



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

K. J. Somaiya School of Engineering, Mumbai-77

(A Constituent College of Somaiya Vidyavihar University)



Department of Electronics Engineering

Course Name:	Object Oriented Programming	Semester:	III
Date of Performance:	___ / ___ / ____	Batch No:	Batch _A-4_
Faculty Name:		Roll No:	16015024082
Faculty Sign & Date:		Grade/Marks:	___/15

Experiment No: 7

Title: I/O Function in Java

Aim and Objective of the Experiment:

Learn the different types of I/O function

COs to be achieved:

CO3: Define exceptions and use I/O streams.

Tools used:

Theory:

1. (Byte stream class , char stream class and different Reading/writing methods)

Code:



(A Constituent College of Somaiya Vidyavihar University)



Department of Electronics Engineering

1. Write a program that lists all the files in a directory including the files present in all its subdirectories as well. Get name/path of the directory from the user through standard input. [Hint: Use recursion] 2. Write a program to read the contents of a file byte by byte and copy it into another file. Get names of the files from the user through standard input. 3. Write a program that appends data to the file using FileWriter class.[Hint: Use a different constructor of FileWriter class]
1. <pre>import java.io.File; import java.util.Scanner; public class ListFilesRecursive { static void walk(File f) { if (f == null !f.exists()) return; System.out.println(f.getAbsolutePath()); if (f.isDirectory()) { File[] kids = f.listFiles(); if (kids != null) for (File k : kids) walk(k); } } public static void main(String[] args) { Scanner sc = new Scanner(system.in); System.out.print("Enter directory path: "); File root = new File(sc.nextLine().trim()); walk(root); } }</pre>
2



(A Constituent College of Somaiya Vidyavihar University)



Department of Electronics Engineering

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Scanner;

public class ByteCopy {
    public static void main(String[] args) {
        Scanner sc = new Scanner(system.in);
        System.out.print("Source file: ");
        String src = sc.nextLine().trim();
        System.out.print("Destination file: ");
        String dst = sc.nextLine().trim();

        try (FileInputStream in = new FileInputStream(src);
            FileOutputStream out = new FileOutputStream(dst)) {
            int b;
            while ((b = in.read()) != -1) out.write(b);
            System.out.println("Copied.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

3



(A Constituent College of Somaiya Vidyavihar University)



Department of Electronics Engineering

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class AppendWithFileWriter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(system.in);
        System.out.print("File to append: ");
        String path = sc.nextLine().trim();
        System.out.print("Text to append: ");
        String text = sc.nextLine();

        try (FileWriter fw = new FileWriter(path, true)) {
            fw.write(text);
            fw.write(System.lineSeparator());
            System.out.println("Appended.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

Post Lab Subjective/Objective type Questions:



(A Constituent College of Somaiya Vidyavihar University)



Department of Electronics Engineering

1. Explain the difference between `FileInputStream` and `BufferedInputStream`. Show an example in support of your answer.

->`FileInputStream` reads bytes directly from a file each time you call `read()`, whereas `BufferedInputStream` wraps another stream and stores data in a buffer to cut down the number of physical read operations, making it generally faster for many small reads.

2. How many lines, words, and characters does a file have? Write a program for the same.

->We can open the file using `BufferedReader`, loop through each line to increment the line counter, split each line on whitespace to count words, and add the character count of each line to get the total characters.

3. Explain in detail all the possible ways of taking inputs from the user.

->User input can be read using `Scanner` on [system.in](#), `BufferedReader` with `InputStreamReader(system.in)` for fast text input, `Console` for interactive console input, command-line arguments through `String[] args`, or dialog boxes like `JOptionPane` for GUI prompts.

Conclusion:

This experiment demonstrates core Java I/O: directory traversal with recursion, byte-level file copying, and file appending using character streams. It also clarifies buffered vs unbuffered reading, common input techniques, and practical file statistics—establishing a solid base for robust, performant file and console I/O in Java.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

K. J. Somaiya School of Engineering, Mumbai-77

(A Constituent College of Somaiya Vidyavihar University)



Department of Electronics Engineering

Signature of faculty in-charge with Date: